

Voice Agent Architecture Whitepaper

A technical architecture guide for realtime customer support systems, voice assistants, and tool-using agents.

Prepared from the uploaded draft and expanded into a startup-style engineering document.

This document extends the original architecture overview with the highest-value sections for production voice systems: call lifecycle, latency budgets, VAD, agent state machines, prompt architecture, escalation paths, and enterprise-grade platform concerns.

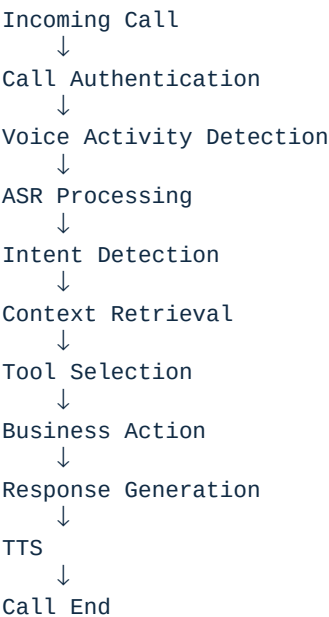
Executive Summary

Voice agents are constrained by latency, state management, prompt composition, and operational reliability. A production system must coordinate speech input, real-time inference, tool execution, retrieval, escalation, and response synthesis while preserving turn-taking behavior and predictable costs.

The architecture below treats the voice assistant as a stateful platform rather than a single model call. That framing makes the system easier to reason about, debug, benchmark, and scale.

1. Detailed Call Lifecycle

The runtime flow should be explicit so engineers can map each user turn to the correct subsystem. This is the control path most teams need when diagnosing barge-in behavior, tool-call delays, or missed responses.



- Call authentication should confirm tenant, caller identity, and policy scope before the model starts acting.
- VAD gates the rest of the pipeline so the system does not pay the ASR and LLM cost during silence.
- Tool selection should be separated from response generation so the agent can decide whether to act, ask a question, or escalate.

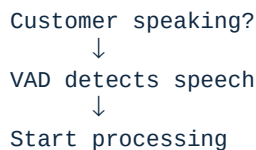
2. Latency Breakdown

Voice products succeed or fail on perceived responsiveness. The latency budget below is a practical target for conversational quality; every stage should report its measured p50 and p95 timings.

Component	Target	Notes
ASR	100 ms	Streaming transcription should start producing partial text quickly.
LLM	200 ms	Keep the prompt small and use the minimum necessary context.
Tool Calls	100 ms	Prefer cached, indexed, or pre-fetched lookups for common paths.
TTS	150 ms	Start synthesis before the whole response is finalized when possible.
Network	100 ms	WebSocket and edge placement matter in live call scenarios.
Total	< 700 ms	A good conversational target for realtime voice startups.

3. Voice Activity Detection

VAD is the gatekeeper for turn-taking. It prevents false starts, supports barge-in, and allows the platform to decide when to transcribe and when to stay idle.

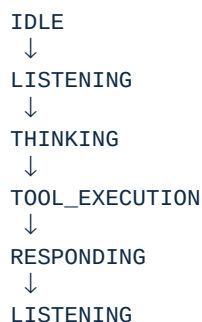


- Silero VAD is a strong default for low-latency speech detection in application code.
- WebRTC VAD is lightweight and often used where CPU budget is tight.
- Deepgram endpointing can be useful when the transcription stack and endpointing policy need to be aligned.

For barge-in systems, the VAD layer should be paired with audio interruption rules so a new customer utterance can cut off the current TTS response cleanly.

4. Agent State Machine

A small state machine makes production behavior understandable. It also creates a common language for logs, traces, retries, and QA playback.



- Transitions should be emitted into telemetry so every turn can be replayed later.

- Tool execution should be a dedicated state, not just an inline function call.
- The system should return to LISTENING after a response so turn-taking stays explicit.

5. Prompt Engineering Layer

At scale, prompt architecture becomes a platform concern. Separate the static policy prompt from runtime context, and keep tool definitions structured so they can be versioned independently.

System Prompt

You are a customer support agent.

Dynamic Prompt

Customer Name
Order History
Current Context

Tool Definitions

```
{
  "name": "check_order"
}
```

The main design rule is to keep policy, state, and factual context separate so prompt changes do not destabilize behavior.

6. AI Agent Builder Architecture

A commercial agent platform is a composition system. The platform must let builders attach prompts, voice, knowledge, tools, policies, and escalation rules without rewriting the runtime.

```
Agent
  ■■■ Prompt
  ■■■ Voice
  ■■■ Knowledge Base
  ■■■ Tools
  ■■■ Policies
  ■■■ Escalation Rules
```

- This is the product shape used by voice-first platforms such as Vapi, Retell, and Bland.
- The builder should store artifacts versioned by tenant and environment.
- Runtime policy evaluation should happen before a risky action or escalation.

7. Human Escalation System

Enterprise voice systems need a clear path from autonomous handling to human takeover. Escalation should be driven by sentiment, confidence, policy violations, and resolution thresholds.

```
Customer angry
  ↓
Sentiment score
  ↓
Escalate to human agent
  ↓
Transfer context and transcript
```

- Preserve transcript, intent, and tool history so the human agent does not start cold.
- Escalation should be auditable and policy-driven rather than ad hoc.
- A handoff should also freeze the agent state to prevent duplicate actions.

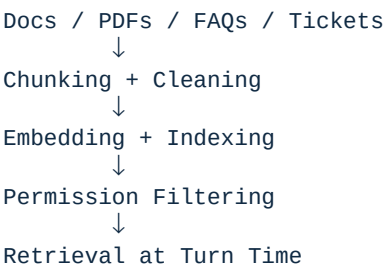
8. Deployment Architecture

The deployment layout should separate realtime media handling, orchestration, storage, and analytics. That separation keeps low-latency paths short and lets batch systems scale independently.

- Realtime media service for WebSocket audio ingress and egress.
- Orchestration service for prompts, routing, tool selection, and policy checks.
- Storage layer for transcripts, call summaries, and audit records.
- Analytics pipeline for quality, latency, and cost tracking.

9. Knowledge Base Ingestion Pipeline

Retrieval quality matters as much as model quality. A strong ingestion pipeline converts documents into searchable, chunked, and permission-scoped context that can be injected into the prompt only when needed.



- Store chunk provenance so the assistant can cite where an answer came from.
- Apply tenant and role filters before retrieval results reach the model.
- Refresh high-priority documents often enough to avoid stale answers.

10. Multi-Tenant SaaS Design

A voice agent platform quickly becomes multi-tenant. Tenancy should be enforced in storage, prompt assembly, tool access, analytics, and billing so customers remain isolated by design.

- Tenant-specific prompts and voice settings should be stored separately from shared infrastructure settings.
- Usage counters must be attributable per tenant and per agent.
- Administrative operations should carry explicit tenant context in every API call.

11. Billing Architecture

Billing in voice systems is usually a combination of minute-based usage, ASR/TTS costs, LLM token costs, and premium feature fees. The platform should compute usage from immutable events rather than mutable dashboards.

Billable Unit	Source	Example
Call Minutes	Call session events	12.4 minutes handled
ASR Usage	Transcription events	37,000 audio frames processed
LLM Tokens	Prompt and response logs	18k input / 4k output tokens
Escalations	Handoff events	3 human transfers

12. Security Deep Dive

Security is not only authentication. Voice systems need tenant isolation, PII handling, prompt-injection defenses, tool authorization, and audit logging for every action that could affect a customer account.

- Authenticate the caller and validate the tenant before loading sensitive context.
- Sanitize external text before it is placed into prompt context or tool parameters.
- Use allowlisted tools with per-tool policy checks.
- Log all customer-impacting operations with trace IDs and actor context.

13. Competitive Analysis

The ecosystem can be grouped by platform emphasis. The architectural tradeoffs below help buyers compare voice-first services quickly.

Platform	Strength	Typical Positioning
Vapi	Developer-first	Fast iteration and flexible integrations
Retell	Voice quality	Conversation handling and production workflows
Bland	Outbound calling	Sales and outreach automation
OpenAI Realtime	Intelligence	Low-latency multimodal agent building
Deepgram	Speech stack	ASR, endpointing, and voice infrastructure

14. System Design Interview Section

This document also works as an interview prep artifact. A strong systems answer should show how the platform handles realtime media, state, scale, and failure modes.

- Scaling WebSockets for many live conversations.
- Real-time audio processing and buffering.
- Distributed queues for async work and retries.
- Failover strategies for ASR, LLM, and TTS providers.
- Cost optimization across inference and telephony.

Key Takeaway

The highest-impact additions are the ones that make the system operationally legible: call lifecycle, latency budget, VAD, agent state machine, prompt architecture, escalation, deployment boundaries, retrieval pipeline, tenancy, billing, security, and competitive positioning.

Taken together, those sections elevate the document from an overview into a whitepaper that reads like a real startup engineering artifact.